Business Profile Extension - SQL Reference & Migration Guide

Pre-Migration Checklist

1. Backup Current Database

```
# Full backup
mysqldump -u root -proot ims > ims_backup_before_business_profiles.sql

# Schema only
mysqldump -u root -proot --no-data ims > ims_schema_backup.sql

# Data only (for restoration testing)
mysqldump -u root -proot --no-create-info ims > ims_data_backup.sql
```

2. Verify Current State

```sql
-- Check current tables
SHOW TABLES;

-- Verify existing relationships
SELECT
    TABLE_NAME,
    CONSTRAINT_NAME,
    CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'ims';

-- Check data counts
SELECT 'users' as tbl, COUNT(*) as cnt FROM users
UNION ALL
SELECT 'vendor_profiles', COUNT(*) FROM vendor_profiles
UNION ALL
SELECT 'investor_profiles', COUNT(*) FROM investor_profiles
UNION ALL
SELECT 'retailer_profiles', COUNT(*) FROM retailer_profiles;
```

Migration Steps

Step 1: Start Application

When you start the Spring Boot application, Flyway will automatically:

1. Create `flyway_schema_history` table

2. Run V1_0__baseline.sql (baseline existing schema)
3. Run V1_1__business_profile_extension.sql (new tables)

Step 2: Verify Migration Success

```sql
-- Check Flyway history
SELECT
    installed_rank,
    version,
    description,
    type,
    script,
    success,
    execution_time
FROM flyway_schema_history
ORDER BY installed_rank;

-- Expected output:
-- | installed_rank | version | description                    | success |
-- |----------------|---------|--------------------------------|---------|
-- | 1              | 0       | Flyway baseline                | 1       |
-- | 2              | 1.0     | baseline                       | 1       |
-- | 3              | 1.1     | business profile extension     | 1       |
```

Step 3: Verify New Tables

```sql
-- List new tables
SHOW TABLES LIKE '%business%';
SHOW TABLES LIKE '%bank%';
SHOW TABLES LIKE '%verification%';

-- Expected output:
-- business_profiles
-- bank_details
-- verification_logs

-- Verify structure
DESCRIBE business_profiles;
DESCRIBE bank_details;
DESCRIBE verification_logs;
```

Step 4: Verify Indexes

```sql
-- Business Profiles indexes
SHOW INDEX FROM business_profiles;

-- Expected indexes:
```

```sql
-- PRIMARY (id)
-- UK_business_profiles_gstin (gstin) - UNIQUE
-- IDX_business_profiles_user_id (user_id)
-- IDX_business_profiles_pan_number (pan_number)
-- IDX_business_profiles_verification_status (verification_status)
-- IDX_business_profiles_gstin (gstin)
-- IDX_business_profiles_user_active (user_id, verification_status)
-- FK_business_profiles_user_id (user_id)

-- Bank Details indexes
SHOW INDEX FROM bank_details;

-- Verification Logs indexes
SHOW INDEX FROM verification_logs;
```

Step 5: Verify Foreign Keys

```sql
SELECT
    CONSTRAINT_NAME,
    TABLE_NAME,
    COLUMN_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME,
    DELETE_RULE,
    UPDATE_RULE
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'ims'
  AND TABLE_NAME IN ('business_profiles', 'bank_details', 'verification_logs')
  AND REFERENCED_TABLE_NAME IS NOT NULL;

-- Expected foreign keys:
-- FK_business_profiles_user_id: business_profiles.user_id → users.id (CASCADE)
-- FK_bank_details_business_profile_id: bank_details.business_profile_id →
business_profiles.id (CASCADE)
-- FK_verification_logs_business_profile_id: verification_logs.business_profile_id
→ business_profiles.id (CASCADE)
```

Step 6: Test Triggers

```sql
-- Insert test business profile
INSERT INTO business_profiles (
    user_id, legal_business_name, business_type, gstin, pan_number,
    registered_address, state, pincode, verification_status
) VALUES (
    1, 'Test Business Ltd', 'PRIVATE_LIMITED', '29TESTB1234T1Z5',
    'encrypted_pan_value', '123 Test Street', 'Karnataka', '560001', 'DRAFT'
);

SET @test_business_id = LAST_INSERT_ID();
```

```sql
-- Insert first bank account (should be primary)
INSERT INTO bank_details (
    business_profile_id, account_holder_name, account_number,
    ifsc_code, bank_name, is_primary, bank_verification_status
) VALUES (
    @test_business_id, 'Test Business Ltd', 'encrypted_acc_1',
    'SBIN0001234', 'State Bank of India', TRUE, 'UNVERIFIED'
);

-- Insert second bank account as primary (should unmark first)
INSERT INTO bank_details (
    business_profile_id, account_holder_name, account_number,
    ifsc_code, bank_name, is_primary, bank_verification_status
) VALUES (
    @test_business_id, 'Test Business Ltd', 'encrypted_acc_2',
    'HDFC0001234', 'HDFC Bank', TRUE, 'UNVERIFIED'
);

-- Verify only one primary
SELECT
    id, bank_name, is_primary, bank_verification_status
FROM bank_details
WHERE business_profile_id = @test_business_id;

-- Expected: Only second account has is_primary = TRUE

-- Cleanup test data
DELETE FROM business_profiles WHERE id = @test_business_id;
```

Post-Migration Verification Queries

Check Data Integrity

```sql
-- Verify all business profiles have valid users
SELECT COUNT(*) as orphan_profiles
FROM business_profiles bp
LEFT JOIN users u ON bp.user_id = u.id
WHERE u.id IS NULL;
-- Should return 0

-- Verify all bank details have valid business profiles
SELECT COUNT(*) as orphan_bank_details
FROM bank_details bd
LEFT JOIN business_profiles bp ON bd.business_profile_id = bp.id
WHERE bp.id IS NULL;
-- Should return 0

-- Verify all verification logs have valid business profiles
SELECT COUNT(*) as orphan_logs
FROM verification_logs vl
```

```
LEFT JOIN business_profiles bp ON vl.business_profile_id = bp.id
WHERE bp.id IS NULL;
-- Should return 0
```

Check Constraints

```
-- Verify ENUM values
SELECT DISTINCT business_type FROM business_profiles;
-- Should only return: PROPRIETORSHIP, LLP, PRIVATE_LIMITED, PARTNERSHIP

SELECT DISTINCT verification_status FROM business_profiles;
-- Should only return: DRAFT, PENDING, VERIFIED, REJECTED

SELECT DISTINCT onboarding_stage FROM business_profiles;
-- Should only return: TIER1_COMPLETE, TIER2_COMPLETE, ACTIVE (or NULL)

SELECT DISTINCT bank_verification_status FROM bank_details;
-- Should only return: UNVERIFIED, VERIFIED, FAILED

SELECT DISTINCT verification_type FROM verification_logs;
-- Should only return: GST, PAN, BANK, CIN, UDYAM
```

Performance Verification

```
-- Test index usage
EXPLAIN SELECT * FROM business_profiles WHERE user_id = 1;
-- Should show: key = IDX_business_profiles_user_id

EXPLAIN SELECT * FROM business_profiles WHERE gstin = '29TESTB1234T1Z5';
-- Should show: key = UK_business_profiles_gstin or IDX_business_profiles_gstin

EXPLAIN SELECT * FROM bank_details WHERE business_profile_id = 1 AND is_primary =
TRUE;
-- Should show: key = IDX_bank_details_is_primary or
IDX_bank_details_business_profile_id
```

Sample Data for Testing

Insert Test Business Profiles

```
-- Insert for existing user (replace user_id = 1 with actual user)
INSERT INTO business_profiles (
    user_id, legal_business_name, business_type, gstin, pan_number,
    registered_address, state, pincode, verification_status, onboarding_stage
) VALUES
(1, 'FlowTrack Solutions Pvt Ltd', 'PRIVATE_LIMITED', '29FLOWS1234S1Z5',
```

```
 'ABCDE1234F', '123 MG Road, Bangalore', 'Karnataka', '560001',
 'VERIFIED', 'ACTIVE'),

(1, 'FlowTrack Retail LLP', 'LLP', '29FLOWR1234R1Z5',
 'FGHIJ5678K', '456 Brigade Road, Bangalore', 'Karnataka', '560002',
 'PENDING', 'TIER2_COMPLETE');

-- Get business IDs
SELECT id, legal_business_name FROM business_profiles WHERE user_id = 1;
```

Insert Test Bank Accounts

```
-- For first business (replace @business_id_1)
SET @business_id_1 = 1; -- Replace with actual ID

INSERT INTO bank_details (
    business_profile_id, account_holder_name, account_number,
    ifsc_code, bank_name, branch_name, is_primary, bank_verification_status
) VALUES
(@business_id_1, 'FlowTrack Solutions Pvt Ltd', '1234567890123456',
 'SBIN0001234', 'State Bank of India', 'MG Road Branch', TRUE, 'VERIFIED'),

(@business_id_1, 'FlowTrack Solutions Pvt Ltd', '9876543210987654',
 'HDFC0001234', 'HDFC Bank', 'Brigade Road Branch', FALSE, 'UNVERIFIED');
```

Insert Test Verification Logs

```
INSERT INTO verification_logs (
    business_profile_id, verification_type, verification_result,
    verification_provider, external_reference_id, ip_address
) VALUES
(@business_id_1, 'GST', 'SUCCESS', 'GSTN API', 'GST_REF_12345', '192.168.1.1'),
(@business_id_1, 'PAN', 'SUCCESS', 'NSDL API', 'PAN_REF_67890', '192.168.1.1'),
(@business_id_1, 'BANK', 'SUCCESS', 'Razorpay', 'RZP_VERIFY_12345',
'192.168.1.1');
```

Common Queries for Operations

1. Find Businesses Needing Verification

```
SELECT
    bp.id,
    bp.legal_business_name,
    bp.gstin,
    bp.verification_status,
    bp.created_at,
```

```
    DATEDIFF(NOW(), bp.created_at) as days_pending
FROM business_profiles bp
WHERE bp.verification_status = 'PENDING'
ORDER BY bp.created_at ASC;
```

2. Find Businesses Without Primary Bank Account

```
SELECT
    bp.id,
    bp.legal_business_name,
    COUNT(bd.id) as total_accounts,
    SUM(CASE WHEN bd.is_primary THEN 1 ELSE 0 END) as primary_accounts
FROM business_profiles bp
LEFT JOIN bank_details bd ON bp.id = bd.business_profile_id
WHERE bp.verification_status != 'DRAFT'
GROUP BY bp.id, bp.legal_business_name
HAVING primary_accounts = 0;
```

3. Verification Success Rate

```
SELECT
    verification_type,
    COUNT(*) as total_attempts,
    SUM(CASE WHEN verification_result = 'SUCCESS' THEN 1 ELSE 0 END) as
successful,
    ROUND(100.0 * SUM(CASE WHEN verification_result = 'SUCCESS' THEN 1 ELSE 0 END)
/ COUNT(*), 2) as success_rate
FROM verification_logs
GROUP BY verification_type;
```

4. User Business Profile Summary

```
SELECT
    u.username,
    u.email,
    COUNT(bp.id) as total_businesses,
    SUM(CASE WHEN bp.verification_status = 'VERIFIED' THEN 1 ELSE 0 END) as
verified_businesses,
    SUM(CASE WHEN bp.verification_status = 'PENDING' THEN 1 ELSE 0 END) as
pending_businesses
FROM users u
LEFT JOIN business_profiles bp ON u.id = bp.user_id
GROUP BY u.id, u.username, u.email;
```

5. Bank Verification Status

```sql
SELECT
    bp.legal_business_name,
    bd.bank_name,
    bd.is_primary,
    bd.bank_verification_status,
    bd.last_verified_at
FROM business_profiles bp
JOIN bank_details bd ON bp.id = bd.business_profile_id
WHERE bp.verification_status = 'VERIFIED'
ORDER BY bp.legal_business_name, bd.is_primary DESC;
```

Troubleshooting

Issue: Migration Failed

```sql
-- Check Flyway status
SELECT * FROM flyway_schema_history WHERE success = 0;

-- Repair Flyway (if schema manually corrected)
-- Run from application: ./mvnw flyway:repair

-- Or manually:
-- DELETE FROM flyway_schema_history WHERE version = '1.1' AND success = 0;
```

Issue: Duplicate Primary Accounts

```sql
-- Find duplicates
SELECT
    business_profile_id,
    COUNT(*) as primary_count
FROM bank_details
WHERE is_primary = TRUE
GROUP BY business_profile_id
HAVING COUNT(*) > 1;

-- Fix: Keep first, unmark rest
UPDATE bank_details bd1
JOIN (
    SELECT
        business_profile_id,
        MIN(id) as keep_id
    FROM bank_details
    WHERE is_primary = TRUE
    GROUP BY business_profile_id
) bd2 ON bd1.business_profile_id = bd2.business_profile_id
SET bd1.is_primary = FALSE
```

```
WHERE bd1.is_primary = TRUE
  AND bd1.id != bd2.keep_id;
```

Issue: Orphaned Records

```
-- Find orphaned bank_details
SELECT bd.*
FROM bank_details bd
LEFT JOIN business_profiles bp ON bd.business_profile_id = bp.id
WHERE bp.id IS NULL;

-- Delete orphaned records (if safe)
DELETE bd FROM bank_details bd
LEFT JOIN business_profiles bp ON bd.business_profile_id = bp.id
WHERE bp.id IS NULL;
```

Rollback Procedure (Emergency Only)

⚠ WARNING: Only use in emergency situations

```
-- 1. Backup current state
mysqldump -u root -proot ims > ims_before_rollback.sql

-- 2. Drop new tables
DROP TABLE IF EXISTS verification_logs;
DROP TABLE IF EXISTS bank_details;
DROP TABLE IF EXISTS business_profiles;

-- 3. Remove Flyway history
DELETE FROM flyway_schema_history WHERE version IN ('1.0', '1.1');

-- 4. Restore from backup
-- mysql -u root -proot ims < ims_backup_before_business_profiles.sql

-- 5. Update application.properties
-- spring.flyway.enabled=false
-- spring.jpa.hibernate.ddl-auto=update
```

Monitoring Queries

Table Sizes

```
SELECT
    table_name,
    ROUND(((data_length + index_length) / 1024 / 1024), 2) as size_mb,
    table_rows
```

```
FROM information_schema.TABLES
WHERE table_schema = 'ims'
  AND table_name IN ('business_profiles', 'bank_details', 'verification_logs')
ORDER BY (data_length + index_length) DESC;
```

Index Usage (MySQL 5.7+)

```
SELECT
    object_schema,
    object_name,
    index_name,
    count_star
FROM performance_schema.table_io_waits_summary_by_index_usage
WHERE object_schema = 'ims'
  AND object_name IN ('business_profiles', 'bank_details', 'verification_logs')
ORDER BY count_star DESC;
```

Recent Activity

```
-- Recent business profile creations
SELECT DATE(created_at) as date, COUNT(*) as new_profiles
FROM business_profiles
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY DATE(created_at)
ORDER BY date DESC;

-- Recent verifications
SELECT DATE(created_at) as date, verification_type, COUNT(*) as attempts
FROM verification_logs
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY DATE(created_at), verification_type
ORDER BY date DESC, verification_type;
```

Maintenance Tasks

Archive Old Verification Logs (Optional)

```
-- Create archive table
CREATE TABLE verification_logs_archive LIKE verification_logs;

-- Archive logs older than 1 year
INSERT INTO verification_logs_archive
SELECT * FROM verification_logs
WHERE created_at < DATE_SUB(NOW(), INTERVAL 1 YEAR);

-- Delete archived logs
```

```
DELETE FROM verification_logs
WHERE created_at < DATE_SUB(NOW(), INTERVAL 1 YEAR);
```

Reindex (If Performance Degrades)

```
OPTIMIZE TABLE business_profiles;
OPTIMIZE TABLE bank_details;
OPTIMIZE TABLE verification_logs;
```

References

- Flyway Documentation: https://flywaydb.org/documentation/
- MySQL ON DELETE CASCADE: https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html
- MySQL Triggers: https://dev.mysql.com/doc/refman/8.0/en/trigger-syntax.html

---

Last Updated: 2026-02-25
Database Version: MySQL 9.2.0
Migration Version: V1.1